

## **Fase 5 - Evaluación Final POA**

Carlos Fernando Rodriguez Salazar

Mayo 2026.

**Universidad Nacional abierta y a distancia-UNAD**  
**Escuela de Ciencias Básicas, Tecnología e Ingeniería – ECBTI**  
**Fundamentos de Programación**

## Introducción

El presente documento expone la solución práctica y argumentativa del problema de evaluación final del curso Fundamentos de Programación. Se aborda la gestión y análisis de una matriz de datos mediante la implementación del paradigma estructurado en el lenguaje Python. Adicionalmente, el desarrollo integra herramientas de uso profesional como el control de versiones a través de Git y GitHub, consolidando así las competencias técnicas fundamentales para el diseño de soluciones lógicas.

## Objetivos

Evaluar el nivel de compromiso de diferentes sesiones de clientes procesando los datos almacenados en una estructura de matriz bidimensional.

Implementar funciones lógicas en Python que clasifiquen parámetros específicos mediante el uso de condicionales compuestos como **and**, y **or**.

Aplicar la versión de código mediante un repositorio público en GitHub como evidencia de las buenas prácticas de desarrollo.

**Código fuente Problema 1:**

```
# Nombre del Estudiante: Carlos Fernando Rodriguez Salazar
# Grupo: 986
# Programa: Ingeniería de Sistemas
# Código Fuente: Autoría propia

def clasificar_sesion(duracion, clics):
    #Clasificar como "Alto" si Duración > 180s y Clics > 8
    if duracion > 180 and clics > 8:
        return "Alto"
    #Clasificar como "Bajo" si Duración < 60s o Clics < 3
    elif duracion < 60 or clics < 3:
        return "Bajo"
    #Clasificar como "Medio" en cualquier otro caso
    else:
        return "Medio"

def generar_informe(matriz_sesiones):
    print("\n" + "="*40)
    print("    REPORTE DE SESIONES")
    print("="*40)
    print(f"{'ID Cliente':<15} | {'Clasificación'}")
    print("-"*40)

    for sesion in matriz_sesiones:
        id_cliente = sesion[0]
        duracion = sesion[1]
        clics = sesion[2]

        clasificacion = clasificar_sesion(duracion, clics)
        print(f"{'id_cliente':<15} | {'clasificacion'}")
    print("="*40 + "\n")

if __name__ == "__main__":
    # Matriz con 5 filas de datos: [ID Cliente, Duración, Clics]
    matriz_sesiones = [
        ["CLI-001", 200, 10], # Alto
        ["CLI-002", 45, 5],   # Bajo (por duración)
        ["CLI-003", 120, 6],  # Medio
        ["CLI-004", 190, 2],  # Bajo (por clics)
        ["CLI-005", 150, 9]   # Medio
    ]

    generar_informe(matriz_sesiones)
```

**Enlace de repositorio GitHub:**

**[Link Repositorio GitHub](#)**

**Enlace del video:**

**[Link Videos](#)**

## Conclusiones

El uso de matrices bidimensionales y estructuras repetitivas for permite procesar grandes volúmenes de datos de manera secuencial y eficiente, separando la lógica de evaluación en funciones independientes que mejoran la legibilidad del código.

La integración de Git y GitHub al flujo de trabajo académico proporciona una trazabilidad clara sobre las versiones del proyecto, siendo una competencia técnica indispensable para colaborar y desplegar código en entornos reales de desarrollo de software.

## Bibliografía

Ejemplo:

Ceballos Sierra, F. J. (2015). C/C++. Curso de programación. 4ª

Edición. <https://elibro->

[net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106454](https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106454)

Deitel, P., & Deitel, H. (2016). Cómo programar en C/C++ . Pearson

Educación. Disponible en Biblioteca Virtual UNAD:

<https://www-ebooks7-24->

[com.bibliotecavirtual.unad.edu.co/?il=16931&pg=145](https://www-ebooks7-24-com.bibliotecavirtual.unad.edu.co/?il=16931&pg=145)

GitHub, Inc. (2025). Acerca de GitHub y Git. GitHub Docs .

<https://docs.github.com/es/get-started/start-your->

[journey/about-github-and-git](https://docs.github.com/es/get-started/start-your-journey/about-github-and-git)